

Data Analysis with Python 2024–2025

Parte 1: Python Soluções dos Exercícios

Soluções independentes baseadas nos enunciados do PDF fornecido

Nota Importante

Este material é uma adaptação das soluções independentes para os exercícios baseados no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**. As soluções abaixo não são o gabarito oficial do curso.

Conteúdo

1 Soluções - Capítulo 1

Exercício 1.1 – hello world

Solução proposta

```
print("Hello, world!")
```

Exercício 1.2 – compliment

Solução proposta

```
country = input("What country are you from? ")  
print(f"I have heard that {country} is a beautiful country.")
```

Exercício 1.3 – multiplication

Solução proposta

```
for i in range(11):  
    print(f"4 multiplied by {i} is {4*i}")
```

Exercício 1.4 – multiplication table

Solução proposta

```
for i in range(1, 11):  
    for j in range(1, 11):  
        print(f"{i*j:4}", end="")  
    print()
```

Exercício 1.5 – two dice

Solução proposta

```
for a in range(1, 7):  
    for b in range(1, 7):  
        if a + b == 5:  
            print((a, b))
```

Exercício 1.6 – triple square

Solução proposta

```
def triple(x):
    return 3*x

def square(x):
    return x*x

for i in range(1, 11):
    if square(i) > triple(i):
        break
    print(f"triple({i})=={triple(i)} square({i})=={square(i)}")
```

Exercício 1.7 – areas of shapes

Solução proposta

```
import math

while True:
    shape = input("Choose a shape (triangle, rectangle, circle): ")
    if shape == "":
        break

    if shape == "triangle":
        base = float(input("Give base of the triangle: "))
        height = float(input("Give height of the triangle: "))
        area = base * height / 2
    elif shape == "rectangle":
        width = float(input("Give width of the rectangle: "))
        height = float(input("Give height of the rectangle: "))
        area = width * height
    elif shape == "circle":
        radius = float(input("Give radius of the circle: "))
        area = math.pi * radius * radius
    else:
        print("Unknown shape!")
        continue

    print(f"The area is {area:f}")
```

Exercício 1.8 – solve quadratic

Solução proposta

```
from math import sqrt

def solve_quadratic(a, b, c):
    d = b*b - 4*a*c
    return ((-b + sqrt(d)) / (2*a), (-b - sqrt(d)) / (2*a))
```

Exercício 1.9 – merge

Solução proposta

```
def merge(L1, L2):
    result = []
    i = j = 0
    while i < len(L1) and j < len(L2):
        if L1[i] <= L2[j]:
            result.append(L1[i])
            i += 1
        else:
            result.append(L2[j])
            j += 1
    result.extend(L1[i:])
    result.extend(L2[j:])
    return result
```

Exercício 1.10 – detect ranges

Solução proposta

```
def detect_ranges(L):
    values = sorted(L)
    result = []
    i = 0
    while i < len(values):
        start = values[i]
        end = start
        while i + 1 < len(values) and values[i+1] == values[i] + 1:
            i += 1
            end = values[i]
        if end > start:
            result.append((start, end + 1))
        else:
            result.append(start)
        i += 1
    return result
```

Exercício 1.11 – interleave

Solução proposta

```
def interleave(*lists):
    result = []
    for group in zip(*lists):
        result.extend(group)
    return result
```

Exercício 1.12 – distinct characters

Solução proposta

```
def distinct_characters(L):  
    return {s: len(set(s)) for s in L}
```

Exercício 1.13 – reverse dictionary

Solução proposta

```
def reverse_dictionary(d):  
    result = {}  
    for english, finnish_words in d.items():  
        for finnish in finnish_words:  
            result.setdefault(finnish, []).append(english)  
    return result
```

Exercício 1.14 – find matching

Solução proposta

```
def find_matching(L, search):  
    return [i for i, value in enumerate(L) if search in value]
```

Exercício 1.15 – two dice comprehension

Solução proposta

```
pairs = [(a, b) for a in range(1, 7) for b in range(1, 7) if a + b  
          == 5]  
for pair in pairs:  
    print(pair)
```

Exercício 1.16 – transform

Solução proposta

```
def transform(s1, s2):  
    a = map(int, s1.split())  
    b = map(int, s2.split())  
    return [x*y for x, y in zip(a, b)]
```

Exercício 1.17 – positive list

Solução proposta

```
def positive_list(L):  
    return list(filter(lambda x: x > 0, L))
```

Exercício 1.18 – acronyms

Solução proposta

```
import string  
  
def acronyms(text):  
    result = []  
    for word in text.split():  
        cleaned = word.strip(string.punctuation)  
        if len(cleaned) >= 2 and cleaned.isupper():  
            result.append(cleaned)  
    return result
```

Exercício 1.19 – sum equation

Solução proposta

```
def sum_equation(L):  
    if not L:  
        return "0 = 0"  
    return " + ".join(map(str, L)) + f" = {sum(L)}"
```

Exercício 1.20 – usemodule

triangle.py

Solução proposta

```
# Funcoes para triangulos retangulos.  
from math import hypot  
  
__version__ = "1.0"  
__author__ = "Autor"  
  
def hypotenuse(a, b):  
    # Retorna o comprimento da hipotenusa.  
    return hypot(a, b)  
  
def area(a, b):  
    # Retorna a area de um triangulo retangulo.  
    return a * b / 2
```

usemodule.py

Solução proposta

```
import triangle

def main():
    print(triangle.hypotenuse(3, 4))
    print(triangle.area(3, 4))

if __name__ == "__main__":
    main()
```